Contribuição: Escudos e correção de velocidade

Oct 6, 2026 · @Marcos

Identificação

Este documento registra a minha contribuição para o jogo Space Invaders da NI1 de Programação Orientada a Objetos. Ela tem duas partes: uma melhoria (escudos de proteção) e a correção de um bug (velocidade do jogo dependente do monitor). O registro segue o modelo do guia do grupo.

1. O que escolhi fazer

Melhoria: escudos (barreiras). Incluí quatro escudos entre o jogador e os inimigos, no formato clássico do Space Invaders. Os escudos bloqueiam os projéteis dos dois lados e escurecem à medida que são danificados. Cada bloco é removido após três impactos, e os inimigos destroem os blocos que alcançam.

Correção de bug: velocidade do jogo. A velocidade do jogo dependia da taxa de atualização do monitor. Em monitores de 144 Hz, o jogo ficava 2,4 vezes mais rápido. Para corrigir, usei um loop de passo fixo, que mantém 60 atualizações por segundo em qualquer monitor.

Avisos ao grupo (regras 1, 4, 5 e 6 do guia):

- O arquivo `Jogo.js` foi alterado. Nenhum outro integrante deve modificá-lo ao mesmo tempo.
- A nova classe `Barreira.js` precisa da tag `<script src="js/Barreira.js"></script>` no `index.html`, depois de `InimigoAtirador.js` e antes de `Tela.js` e `Jogo.js`. A tag já está incluída na minha cópia, para fins de teste. O responsável pelo `index.html` deve incluí-la no projeto final.

2. Uso de IA

Foi enviado ao Claude o projeto em `.zip` (versão com telas de início e fim de jogo) e o PDF do guia do grupo. Os prompts estão transcritos exatamente como foram enviados, conforme orientação do guia.

Prompt 1 (Claude):

> *a gente vai fazer esse trabalho para vc entender, esse segundo arquivo que te mandei foi a parte que o claude fez e e meu amigo melhorou pois estava muito simples, ai queria que me aJUDA AGORA a melhorar esse jogo e me mandar oq vc melhorou*

Resultado: a IA analisou o guia e todo o código do projeto. Ela mediu a velocidade do jogo simulando monitores de 60 Hz e 144 Hz e, com isso, encontrou o bug de velocidade. Em seguida, criou a classe `Barreira` (que herda de `Entidade`), integrou os escudos ao `Jogo.js` e substituiu o

loop por um de passo fixo. Por fim, testou o resultado no navegador, com capturas de tela, uma partida completa até o fim de jogo e a verificação de erros no console.

Prompt 2 e Prompt 3 (Claude):

> *e no primeiro arquivo te mandei para entender otrabalh oq deve ser feito*

> *e seguir oq esta escrito la*

Resultado: a IA revisou a entrega de acordo com as regras do guia. Ela padronizou o nome das capturas de tela, registrou os avisos ao grupo, preencheu este registro conforme o modelo e preparou o pacote de entrega no mesmo formato usado pelo grupo no GitHub.

Segunda ferramenta e comparação: não se aplica. A comparação entre IAs foi feita por outro integrante do grupo.

3. Bug encontrado

Sintoma: em computadores com monitores de 120 ou 144 Hz, os inimigos, o jogador e os projéteis se moviam bem mais rápido. Por isso, o jogo ficava mais difícil do que em um monitor convencional de 60 Hz.

Causa: o `requestAnimationFrame` executa o `_loop()` uma vez por quadro do monitor, ou seja, 60, 120 ou 144 vezes por segundo. Cada execução deslocava os elementos uma quantidade fixa de pixels. Assim, quanto maior a taxa de quadros, mais rápido o jogo ficava.

Medição feita em 0,5 segundo de jogo:

Monitor	Quadros	Deslocamento do inimigo (antes)	Deslocamento do inimigo (depois)
60 Hz	30	30 px	29 px
144 Hz	72	**72 px** (2,4× mais rápido)	29 px

Correção: separei as regras do jogo no método `_atualizar()`. O `_loop(agora)` soma o tempo real que passou e executa `_atualizar()` uma vez a cada 1/60 de segundo acumulado. Essa técnica se chama passo fixo, ou *fixed timestep*. O tempo acumulado é limitado a 250 ms, para que o jogo não avance vários passos de uma vez quando a aba fica em segundo plano.

4. Código antes e depois

4.1 Loop do jogo (`Jogo.js`)

Antes:

```
_loop() {
  if (!this.rodando) return;

  this._tratarInput();
  this.jogador.atualizar();
  this._atualizarInimigos();
  this._atualizarProjeteis();
  this._verificarColisoes();
  this._verificarFimDeJogo();
  this._desenhar();

  if (this.rodando) {
    this.idFrame = requestAnimationFrame(this._loop);
  }
}
```

Depois:

```
// no constructor:
this.passo = 1000 / 60;   // duração de 1 atualização, em milissegundos
this.tempoAcumulado = 0;  // tempo real que ainda falta "simular"
this.ultimoTempo = 0;     // momento do quadro anterior

// no iniciar(), antes do requestAnimationFrame:
this.tempoAcumulado = 0;
this.ultimoTempo = performance.now();

// Uma "rodada" das regras do jogo (equivale a 1/60 de segundo).
_atualizar() {
  this._tratarInput();
  this.jogador.atualizar();
  this._atualizarInimigos();
  this._atualizarProjeteis();
  this._verificarColisoes();
  this._verificarFimDeJogo();
}

_loop(agora) {
  if (!this.rodando) return;

  // Limita a 250 ms para o jogo não avançar demais após a aba ficar em segundo plano
  this.tempoAcumulado += Math.min(agora - this.ultimoTempo, 250);
  this.ultimoTempo = agora;

  while (this.tempoAcumulado >= this.passo && this.rodando) {
    this._atualizar();
    this.tempoAcumulado -= this.passo;
  }

  this._desenhar();

  if (this.rodando) {
    this.idFrame = requestAnimationFrame(this._loop);
  }
}
```

4.2 Escudos (nova classe `Barreira.js`)

Antes: não se aplica. É um arquivo novo; antes dele, o jogador não tinha nenhuma proteção.

Depois:

```
class Barreira extends Entidade {
  constructor(x, y) {
    super(x, y, Barreira.TAMANHO, Barreira.TAMANHO, '#33ccff');
    this.resistencia = 3; // aguenta 3 tiros antes de sumir
  }

  // Chamado quando um projétil atinge este bloco
  receberDano() {
    this.resistencia--;
    if (this.resistencia <= 0) this.ativo = false;
  }

  // Sobrescreve desenhar() da Entidade: a cor escurece com o dano
  desenhar(ctx) {
    const coresPorResistencia = { 3: '#33ccff', 2: '#2389ad', 1: '#174f63' };
    ctx.fillStyle = coresPorResistencia[this.resistencia];
    ctx.fillRect(this.x, this.y, this.largura, this.altura);
  }

  // Monta um escudo a partir de um desenho em texto ('#' = bloco)
  static criarEscudo(xInicial, yInicial) {
    const formato = [
      '..######..',
      '.########.',
      '##########',
      '##########',
      '###....###',
      '##......##',
    ];
    const pedacos = [];
    formato.forEach((linha, l) => {
      [...linha].forEach((caractere, c) => {
        if (caractere === '#') {
          pedacos.push(new Barreira(xInicial + c * Barreira.TAMANHO,
                                    yInicial + l * Barreira.TAMANHO));
        }
      });
    });
    return pedacos;
  }
}
Barreira.TAMANHO = 8;
```

4.3 Integração dos escudos (`Jogo.js`)

```
// _resetarEstado(): cria os escudos a cada partida
this.barreiras = this._criarBarreiras();

// Novo método: 4 escudos espaçados igualmente, na altura y = 460
_criarBarreiras() {
  const larguraEscudo = 10 * Barreira.TAMANHO;
  const quantidade = 4;
  const espaco = this.canvas.width / quantidade;
```

```
  let pedacos = [];
  for (let i = 0; i < quantidade; i++) {
    const x = espaco * i + (espaco - larguraEscudo) / 2;
    pedacos = pedacos.concat(Barreira.criarEscudo(x, 460));
  }
  return pedacos;
}

// _verificarColisoes(): projéteis (dos dois lados) x escudos
[...this.projeteisJogador, ...this.projeteisInimigo].forEach((projetil) => {
  for (const pedaco of this.barreiras) {
    if (projetil.ativo && pedaco.ativo && projetil.colideCom(pedaco)) {
      projetil.ativo = false;
      pedaco.receberDano();
      break;
    }
  }
});
// inimigos x escudos: o inimigo destrói o bloco que tocar
this.inimigos.forEach((inimigo) => {
  this.barreiras.forEach((pedaco) => {
    if (pedaco.ativo && inimigo.colideCom(pedaco)) pedaco.ativo = false;
  });
});
this.barreiras = this.barreiras.filter((b) => b.ativo);

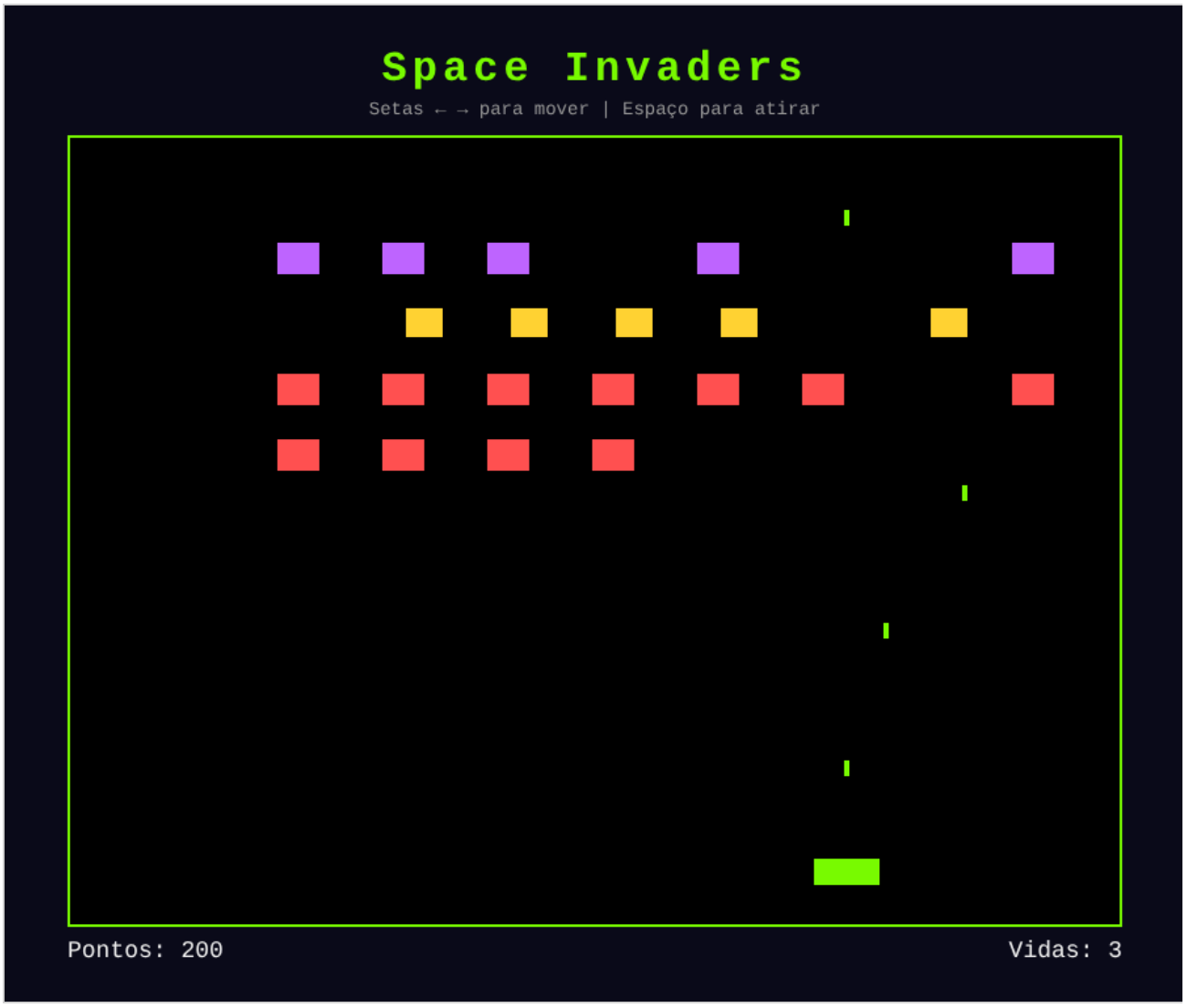
// _desenhar():
this.barreiras.forEach((b) => b.desenhar(ctx));
```

`index.html`: nova linha após `InimigoAtirador.js`:

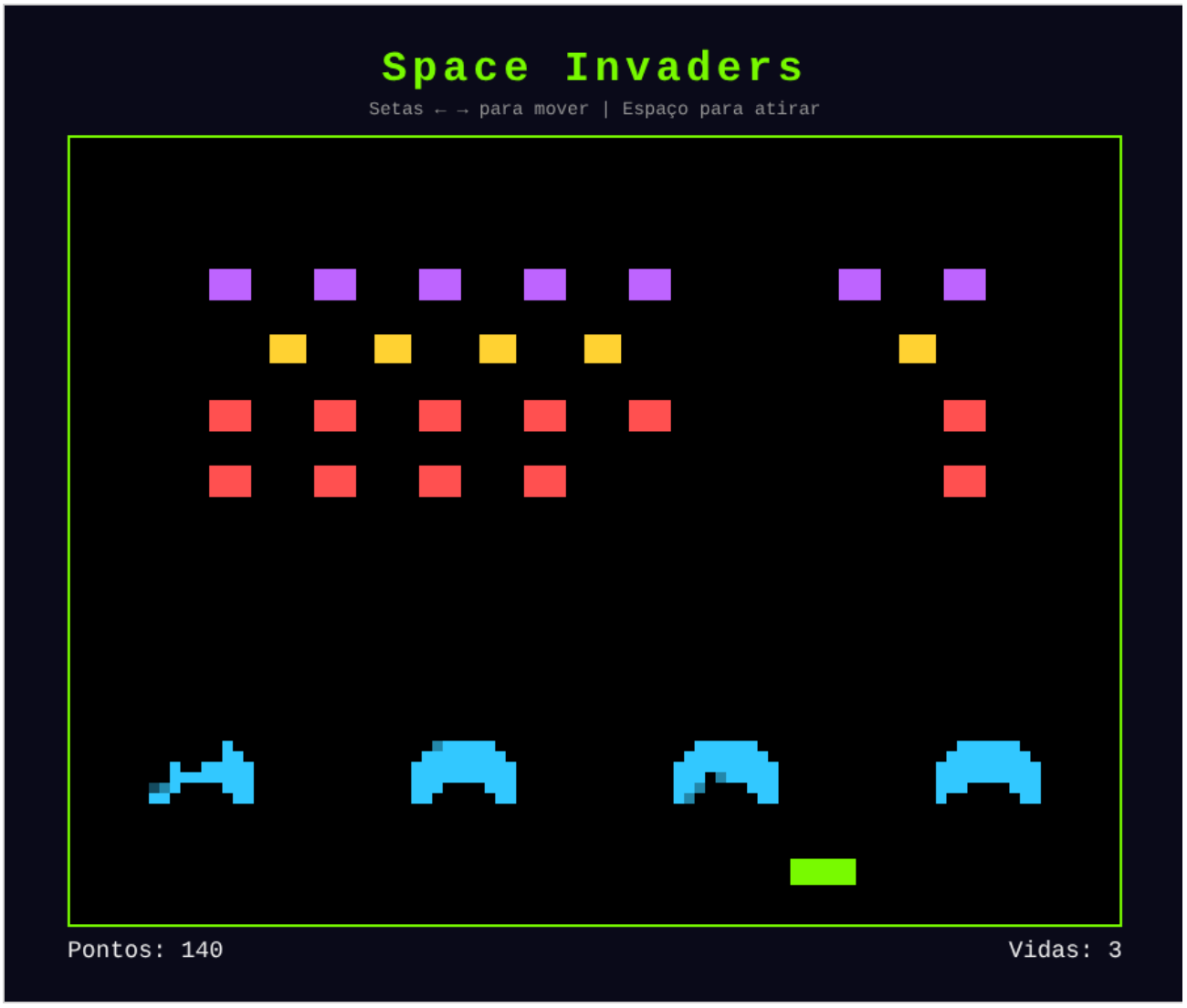
```
<script src="js/Barreira.js"></script>
```

5. Evidências

Antes da alteração (`marcos_antes.png`): o jogo sem escudos.



Depois da alteração (`marcos_depois.png`): o jogo com os quatro escudos. O escudo da esquerda está danificado, com blocos mais escuros.



Bug de velocidade: a tabela de medição está na seção 3 (60 Hz × 144 Hz, antes e depois).

Teste adicional: em uma partida simulada até o fim, os inimigos atravessaram os escudos e a tela "GAME OVER – Os invasores chegaram até você!" apareceu corretamente, sem erros no console.

6. O que entendi e vou explicar na apresentação

Como a minha parte funciona:

- Cada escudo é formado por vários blocos de 8×8 pixels, e cada bloco é um objeto da classe `Barreira`.
- O método estático `criarEscudo()` lê um desenho em texto e cria um objeto para cada caractere `#`.
- Quando um projétil atinge um bloco, o projétil é desativado e o bloco perde um ponto de resistência. Quando a resistência chega a zero, o bloco é removido. Se um inimigo toca um bloco, ele o destrói na hora.
- Quanto ao bug, o jogo passou a contar o tempo real que passou, e não a quantidade de quadros. Por isso, as regras rodam 60 vezes por segundo em qualquer monitor.

Conceitos de POO aplicados:

Conceito	Onde aparece
Herança	`Barreira extends Entidade` reaproveita a posição, o tamanho e o método `colideCom()` sem reescrevê-los
Polimorfismo	`Barreira` sobrescreve `desenhar()` para mudar de cor conforme o dano; o `Jogo` só chama `b.desenhar(ctx)`, sem precisar saber como o bloco é desenhado
Encapsulamento	A regra de dano fica dentro de `receberDano()`; o `Jogo` não altera a resistência diretamente
Método estático	`Barreira.criarEscudo()` pertence à classe, e não a um objeto específico

7. Créditos

Não usei assets nem código de terceiros. O formato dos escudos é inspirado no Space Invaders original (Taito, 1978). A técnica de passo fixo é um padrão consolidado no desenvolvimento de jogos, descrito no artigo “Fix Your Timestep!”, de Glenn Fiedler (https://gafferongames.com/post/fix_your_timestep/).